

SYNTRY FACE RECOGNITION SYSTEM DOCUMENTATION

ALI MADARI

Table of Contents

SyntryRepository.cs – Database Access	7
Purpose.....	7
Class Structure	7
Connection Handling	7
Constructor.....	7
Creating a connection	7
Checking if a User has a Face Data	8
Purpose.....	8
SQL Query	8
Upsert User (Insert or Update).....	8
Purpose.....	8
SQL Query	8
Log Attendance	8
Purpose.....	9
SQL Query	9
Set User Active / Inactive	9
Purpose.....	9
SQL Query (Active : 1 and Disabled : 0).....	9
Delete User.....	9
Purpose.....	10
SQL Query	10
Get Username Helper.....	10
Purpose.....	10
SQL Query	10
Search Users by Name	10
Purpose.....	10
SQL Query	10
Program.cs – Application Entry Point	10
Purpose.....	11
Global Variables.....	11
Purpose.....	11
Purpose.....	11

Purpose.....	11
Main Method.....	11
Read Command Line Port – Step 1.....	11
Purpose.....	11
Configure Logging – Step 2.....	11
Create Host Builder– Step 3	12
Load Configuration– Step 4.....	12
Override Port if CLU Argument Exists.....	12
Dependency Injection– Step 5	13
Load Configuration Values from appsettings.json.....	13
Register Database Repository	13
Enable Serilog for Host– Step 6.....	13
Build the Host– Step 7	13
Run the Server– Step 8	13
Error Handling and Cleaning Logs	14
ServerHost.cs – Core Runtime Orchestrator.....	15
Purpose.....	15
Ready Signal	15
Logger	15
Configuration Access	16
MAIN METHOD.....	16
Initialize AI Models – Step 1	16
Load Face Embeddings – Step 2.....	16
Start Background Synchronization – Step 3	17
Start WebSocket Server – Step 4	17
Notify System that Server is Ready – Step 5.....	17
Error Handling	17
WebSocketLoader.cs – Device Communication & Processing Engine	18
Purpose.....	18
Device Sessions	18
Device Serial Number Mapping	18
Admin Sessions.....	19
Pending Face Enrollments	19

PendingEnrollment Structure.....	19
AI Scan Queue	20
AI Worker Threads.....	20
Starting the WebSocket Server	20
Server Configuration.....	20
Connection and Disconnection Handlers.....	20
Message Handling.....	20
Command Dispatcher.....	21
Device Registration.....	21
Face Enrolment Workflow Enrolment Steps	22
Handling Device Logs	22
Scan Detection	22
AI Scan Processing	22
AI Recognition.....	23
Access Decision Logic.....	23
Live Scan Feed for Admins.....	23
Cleaning Device Logs.....	23
FaceRecognition.cs (FaceMatch) — AI Face Recognition Engine	24
Purpose.....	24
User Information Structure	24
AI Threshold Configuration.....	24
FaceDetectorYN	25
FaceRecognizerSF	25
Model Initialization.....	25
Loading Face Embeddings.....	25
SQL Query	25
Two-Phase Loading Strategy	26
Phase 1 — Fast Database Read.....	26
Phase 2 – AI processing	26
Result Stored in RAM.....	26
Matching a Face.....	26
Face Feature Extraction	26
Liveness Detection.....	27

Face Matching Algorithm	27
Match Result Object	27
Adding a New User to Memory	27
Removing a User.....	28
Database Synchronization.....	28
Antispoofing.cs — Liveness Detection Module	29
Purpose:.....	29
Core Mode	29
Model Initialization.....	29
Prediction Process	29
Processing Steps.....	30
Crop Face Region – Step 1	30
Image Preprocessing – Step 2	30
Neural Network Inference – Step 3	30
Softmax Normalization – Step 4.....	30
Output Result	30
Thread Safety	30
ScanJob.cs — Scan Task Structure	31
Purpose.....	31
Class Definition	31
Fields.....	31
Device Serial Number.....	31
Face Image	31
WebSocket Session	31
Timestamp.....	31
ConsoleInteractor.cs — Console Interaction Utility	32
Purpose.....	32
Key Feature	32
Ask a Question.....	32
Timeout Handling	32
Yes / No Helper.....	32
Thread Safety	32
WebSocketService.cs — Windows Service Wrapper	33

Purpose.....	33
Linux Server Implementation	33
Device Settings	34
System:.....	34
Device Setup:	34
Advanced Setup	35
Access:	36
Shift:.....	37

SynteryRepository.cs – Database Access

Purpose

Is responsible for all database operation related to users and attendance. It acts a database access between the application and the PostgreSQL Server Database. All database logic is mostly centralized inside this class.

Class Structure

```
public class SynteryRepository
```

This class will contain all methods used to interact with the database.

Key features:

- asynchronous database calls
- connection encapsulation
- attendance logging
- user management

Connection Handling

```
private readonly string _connString;
```

Stores the PostgreSQL Server connection string passed from the main application.

Constructor

```
public SynteryRepository(string connString)
```

```
{
```

```
    _connString = connString;
```

```
}
```

The constructor initializes the repository with the database connection string.

Creating a connection

```
private IDbConnection CreateConnection() => new SqlConnection(_connString);
```

This helper method creates a **new SQL connection** whenever a database operation is executed.

Connections are opened inside each method and disposed automatically using the keyword “using”

Checking if a User has a Face Data

```
public async Task<bool> HasFaceDataAsync(int enrollId)
```

Purpose

Checks whether a user already has facial data stored in the database. preventing duplicate enrolment.

SQL Query

```
SELECT COUNT(1)
FROM tblusers_face
WHERE enrollid = @Id AND backupnum = 50
```

Returns:

true → user already has face

false → no face registered

Upsert User (Insert or Update)

```
public async Task UpsertUserAsync(int enrollId, string name, int backupNum,
int isAdmin, string? record)
```

Purpose

Adds a new user or updates an existing user.

INSERT if user does not exist

UPDATE if user already exists

SQL Query

```
MERGE tblusers_face AS target
USING (SELECT @Id AS enrollid, @Num AS backupnum) AS source
```

Returns:

If User Exists Update:

Username , admin ,record (face template) , regdatetime , isactive

If User Does Not Exist:

Insert a new record

Log Attendance

```
public async Task LogAttendanceAsync(int? enrollId, string deviceSn,
DateTime time, double? distance)
```

Purpose

Stores **attendance events** given by face scans.

SQL Query

Preventing Double Scan – Step 1

```
SELECT TOP 1 attendatetime  
FROM tblattendance_face  
WHERE enrollid = @EnrollId  
ORDER BY attendatetime DESC
```

This retrieves the user's last scan time.

Anti Double-Scan Logic

```
if ((time - lastScanTime).TotalSeconds < 20)
```

If the same user scans again within 20 seconds, the scan is ignored preventing duplicate scans.

Insert Attendance Record – Step 2

```
INSERT INTO tblattendance_face (enrollid, device, attendatetime)  
VALUES (@EnrollId, @Device, @Time);
```

Stores the Fields Enroll id, Device Serial Number and timestamp of the scan into Attendance

Fetch Username – Step 3 (For Future API Implementation as Instructed)

```
SELECT username FROM tblusers_face
```

Grabs the username from the database

Set User Active / Inactive

```
public async Task SetUserActiveAsync(int enrollId, bool isActive)
```

Purpose

Enable or disable a user controlling whether a user can access the system.

SQL Query (Active : 1 and Disabled : 0)

```
UPDATE tblusers_face  
SET isactive = @Active  
WHERE enrollid = @Id
```

Delete User

```
public async Task DeleteUserAsync(int enrollId)
```

Purpose

Completely removes a user from the database.

SQL Query

```
DELETE FROM tblusers_face
```

```
WHERE enrollid = @Id
```

Removing all authentication types related to that user.

Get Username Helper

```
public string? GetUserName(int enrollId)
```

Purpose

Retrieve the username using an enrollId.

SQL Query

```
SELECT TOP 1 username
```

```
FROM tblusers_face
```

```
WHERE enrollid = @Id
```

Used by other components to get username of a user.

Search Users by Name

```
public static async Task<List<(int EnrollId, string UserName, int IsActive)>> SearchUsersByNameAsync
```

Purpose

Allows **admin UI to search users** by Name.

SQL Query

```
@ "SELECT enrollid, username, isactive
```

```
FROM tblusers_face
```

```
WHERE username ILIKE @name
```

```
ORDER BY username",
```

Get Users that are alike

Program.cs – Application Entry Point

Is the main entry point of the server application

Purpose

It has multiple functions such as:

- Initializing **logging**
- Loading **configuration**
- Setting up **dependency injection**
- Starting the **background services**
- Launching the **WebSocket server**

Global Variables

```
public static LoggingLevelSwitch ConsoleSwitch = new(LogEventLevel.Information);
```

Purpose

allows **changing console log levels dynamically**. (**Default : Information**)

Global Connection String

```
public static string ConnectionString { get; private set; } = string.Empty;
```

Purpose

stores the database connection string so it can be accessed by other classes

Server Port

```
public static int ServerPort { get; private set; }
```

Purpose

Stores the WebSocket server port loaded from configuration (appsettings.json)

Main Method

Read Command Line Port - Step 1

```
int? cliPort = null;  
  
if (args.Length > 0 && int.TryParse(args[0], out int p))  
    cliPort = p;
```

Purpose

Allows the server port to be **overridden from command line**

Configure Logging - Step 2

```
Log.Logger = new LoggerConfiguration()
```

```
.WriteTo.Console(levelSwitch: ConsoleSwitch)
```

```
.WriteTo.File("logs/syntery-.log", rollingInterval:  
RollingInterval.Day)
```

```
.CreateLogger();
```

This system uses Serilog Logging System, Two Logging outputs are configured:

- Console Logging
 - WriteTo.Console() [Logs are displayed in the terminal]
- File Logging
 - logs/syntery-YYYYMMDD.log [Log files rotate daily]

Create Host Builder - Step 3

```
var host = Host.CreateDefaultBuilder(args)
```

The .NET Generic Host provides:

- dependency injection
- configuration
- background services
- logging
- lifecycle management

Load Configuration - Step 4

```
config.AddJsonFile("appsettings.json", optional: false, reloadOnChange:  
true);
```

Loads Configuration from appsettings.json

- Server:Port
- ConnectionStrings:Default
- AI:FaceDetection
- AI:FaceRecognition
- AI:AntiSpoof
- Face:MatchThreshold
- Face:LivenessThreshold

Override Port if CLU Argument Exists

```
config.AddInMemoryCollection(new Dictionary<string, string?>{  
  
    { "Server:Port", cliPort.Value.ToString() } });
```

Dependency Injection - Step 5

```
.ConfigureServices((context, services) =>
```

registers application services

Load Configuration Values from appsettings.json

```
ConnectionString = context.Configuration.GetConnectionString("Default")!;
```

```
ServerPort = context.Configuration.GetValue<int>("Server:Port");
```

Register Database Repository

```
services.AddSingleton(new SynteryRepository(ConnectionString));
```

creates a single shared instance of the repository.

Register ServerHost Background Service

```
services.AddHostedService<ServerHost>();
```

This is the service that starts the system, Launching:

```
ServerHost.ExecuteAsync();
```

Which:

- Loads AI models
- Loads face embeddings
- Starts the WebSocket server

Enable Serilog for Host - Step 6

```
.UseSerilog();
```

Replaces the default .Net Logger with Serilog

Build the Host - Step 7

```
.Build();
```

This compiles the configuration and services into a runnable application host

Run the Server - Step 8

```
await host.RunAsync();
```

starts the application lifecycle. The host will start the background services and wait for the shutdown signal which is CTRL + C which stops all the services and shuts down the system.

Error Handling and Cleaning Logs

```
try{  
}  
catch (Exception ex)  
{  
    Log.Fatal(ex, "Application start-up failed");  
}
```

If the server fails during the startup, a fatal error will be logged.

```
Log.CloseAndFlush();
```

Ensures all logs are written before the program exits.

ServerHost.cs – Core Runtime Orchestrator

background service responsible for initializing and running the core components of the system.

Purpose

It performs the following critical tasks:

1. Initialize the **AI models** used for face detection and recognition
2. Load **face embeddings from the database into memory**
3. Start a **background synchronization process** with the database
4. Launch the **WebSocket server** that communicates with face devices
5. Signal that the server is **ready to accept connections**

This class inherits from .NET BackgroundService, meaning it runs continuously as part of the application's host lifecycle.

```
public class ServerHost : BackgroundService
```

BackgroundService is a built-in .NET class used to run long-running services inside the Generic Host environment.

Ready Signal

```
public static readonly TaskCompletionSource<bool> ReadySignal = new();
```

This is used to notify other components (such as a console UI or monitoring system) that the server has successfully started having two values:

True – Server started successfully

False – Server failed during initialization

Logger

```
private readonly ILogger<ServerHost> _logger;
```

This logger is injected by the dependency injection container allowing the service to write structured logs

Configuration Access

```
private readonly IConfiguration _config;
```

Provides access to the values stored in Json files, command-line overrides and environment variables

MAIN METHOD

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
```

This method runs continuously while the server is active, executing the initialization sequence

Initialize AI Models – Step 1

```
FaceMatch.InitModels(  
    Path.Combine(baseDir, _config["AI:FaceDetection"]!),  
    Path.Combine(baseDir, _config["AI:FaceRecognition"]!),  
    Path.Combine(baseDir, _config["AI:AntiSpoof"]!),  
    _config.GetValue<double>("Face:MatchThreshold"),  
    _config.GetValue<float>("Face:LivenessThreshold")  
);
```

This loads the deep learning models required for face recognition.

Model	Name	Purpose
Face Detection	face_detection_yunet_2023mar.onnx	locate faces in an image
Face Recognition	face_recognition_sface_2021dec.onnx	generate face embeddings
Anti-Spoof Model	MiniFASNetV2.onnx	detect fake faces

Threshold Configuration

MatchThreshold – Similarity score for matching

LivenessThreshold – Minimum probability to detect fake photos and videos

Load Face Embeddings – Step 2

```
FaceMatch.LoadEmbeddings(_config.GetConnectionString("Default")!);
```

This loads all registered users from the database, The system converts each face image into a **128-dimension embedding vector** and stores it in RAM to speed up matching and reduce database bottlenecking

Start Background Synchronization – Step 3

```
_ = Task.Run(async () =>
{
    while (!stoppingToken.IsCancellationRequested)
    {
        await Task.Delay(TimeSpan.FromSeconds(30), stoppingToken);
        await FaceMatch.SyncByComparisonAsync();
    }
});
```

This background task ensures that RAM data stays synchronized with the database syncing every 30 seconds. This guarantees database remains the source of truth

Case	Action
New User added in Database	Load into RAM
User deleted	Remove from RAM
User active status changed	update RAM

Start WebSocket Server – Step 4

```
int port = _config.GetValue<int>("Server:Port");
await WebSocketLoader.StartServerAsync(port);
```

This launches the WebSocket communication server on the port specified

Notify System that Server is Ready – Step 5

```
_logger.LogInformation("Server is running on port {Port}", port);
ReadySignal.TrySetResult(true);
```

This signals that initialization completed successfully and other components can now safely interact with the server.

Error Handling

```
catch (Exception ex) {
    _logger.LogCritical(ex, "FATAL: Server failed to start.");
    ReadySignal.TrySetResult(false);
}
```

The server logs a critical error and signals failure

```
await Task.Delay(Timeout.Infinite, stoppingToken);
```

This ensures the background service remains active until the application is stopped.

WebSocketLoader.cs – Device Communication & Processing Engine

responsible for managing all WebSocket communication between the server and connected devices or admin clients.

Purpose

It acts as the **central communication hub** of the system.

Main responsibilities:

- Start and manage the **WebSocket server**
- Track **connected devices and admin sessions**
- Handle **incoming device messages**
- Process **face scans**
- Manage **face enrollment**
- Execute **admin commands**
- Distribute AI scan jobs to **worker threads**

Device Sessions

```
private static readonly ConcurrentDictionary<string, WebSocketSession>  
_sessions
```

Stores active WebSocket sessions.

- Key = Session ID
- Value = WebSocket session object

Device Serial Number Mapping

```
private static readonly ConcurrentDictionary<string, string>  
_deviceToSession
```

Maps device serial numbers (SN) to session IDs allowing the server to **send commands directly to a specific device**.

Admin Sessions

```
private static readonly ConcurrentDictionary<string, WebSocketSession>  
adminSessions
```

Stores connected **admin control panels**.

Pending Face Enrollments

```
private static readonly ConcurrentDictionary<string, PendingEnrollment>  
pendingEnrollmentsBySn
```

Tracks devices currently enrolling a user.

Key = device serial number.

This ensures:

- only one enrollment per device
- enrollment timeouts
- correct face capture workflow.

PendingEnrollment Structure

```
private class PendingEnrollment  
{  
    public int EnrollId { get; init; }  
    public string UserName { get; init; } = "";  
    public bool IsAdmin { get; init; }  
    public int ShotsRemaining { get; set; } = 2;  
    public DateTime StartedAt { get; init; } = DateTime.UtcNow;  
}
```

Field	Purpose
EnrollId	User ID being registered
UserName	name of the user
IsAdmin	admin privileges
ShotsRemaining	number of face images still required
StartedAT	enrolment start time

AI Scan Queue

```
private static readonly Channel<ScanJob> _scanQueue
```

This queue stores face scan requests waiting for AI processing with a Max requests of 100 if the queue becomes 100 then the oldest scan is dropped

AI Worker Threads

```
private static readonly int _aiWorkerCount = 2;
```

Number of AI Workers in the system, each worker continuously reads from the scan queue.

Starting the WebSocket Server

```
public static async Task<IHost> StartServerAsync(int port)
```

initializing the WebSocket server

Server Configuration

```
WebSocketHostBuilder.Create()
```

This builds a WebSocket server using SuperSocket

Connection and Disconnection Handlers

```
onConnected and OnClosed
```

Executed when a device connects and when a device is disconnected. When the device disconnects the system removes the session, clears device mapping and cancels any pending enrolment

Message Handling

```
HandleIncomingMessage(WebSocketSession session, string msg)
```

Every incoming message is processed by this handler, and the message should be formatted in JSON

Command Dispatcher

```
var cmd = json.Value<string>("cmd");
```

The server will read this command that is sent to the server and then routes it using the switch statement

Commands that the server is ready for:

Command	Purpose
reg	Device Registration
sendlog	Device Scan Logs
senduser	Face Enrolment Data
admin_add_user	Start Enrolment
admin_delete_user	Remove User
admin_set_active	Activate/Deactivate
admin_get_user	Retrieve User
admin_list_devices	List Connected Devices
admin_ping	
admin_hello	Handles Admin Registration
admin_search_user_by_name	Retrieves Users by Name for Admin
ping	Ping Pong System for Latency
admin_reboot	Reboot Devices by Admin
admin_set_time	Set Time by Admin

Device Registration

```
HandleRegister()
```

When a device connects, it sends:

```
cmd = "reg"
```

The server:

1. registers the session
2. stores the device SN
3. sends time synchronization

Face Enrolment Workflow

Enrolment Steps

1. Admin requests enrolment
2. Server registers PendingEnrollment
3. Device sends captured face images
4. Images stored in database
5. User embedding generated
6. User becomes active in memory

Handling Device Logs

`HandleSendLog()`

function processes logs sent by devices.

Scan Detection

If the device sends :

`note = "face not found"`

and an image is included; the servers create a scan job that is pushed into the AI Queue

`var job = new ScanJob`

```
{  
    DeviceSn = sn,  
    ImageBase64 = img,  
    Session = s,  
    Timestamp = DateTime.UtcNow  
};
```

AI Scan Processing

Each AI Worker processes jobs using:

`ProcessScanJob()`

AI Recognition

The Server runs

```
FaceMatch.MatchFaceFromBase64()
```

Which Returns

Matched	Whether a user matched
EnrollID	user ID
Score	match similarity score
Liveness	anti-spoof result score

Access Decision Logic

If user exists and is active:

ACCESS GRANTED and Device Receives : Welcome "Username"

Then Attendance is Logged in the Database

If user is inactive:

ACCESS DENIED and Device Receives : User Inactive : Username

If no user matched:

User not found

Live Scan Feed for Admins

```
ret: live_scan
```

Allows monitoring from the admin client for all scans as it sends back the result

Cleaning Device Logs

```
cleanlog and cleanuser
```

This clears the device memory to prevent duplicate events and security precautions

FaceRecognition.cs (FaceMatch) — AI Face Recognition Engine

core artificial intelligence component responsible for performing facial recognition.

Purpose

It processes images received from devices and determines whether the face matches a registered user.

The module performs several tasks:

- Detect faces in an image
- Generate face feature embeddings
- Compare embeddings with known users
- Perform **anti-spoof detection**
- Manage an **in-memory database of users**
- Synchronize with the SQL database

Because facial recognition must be extremely fast, this module loads all face embeddings into **RAM**, allowing comparisons to occur in milliseconds.

User Information Structure

```
public class UserInfo
```

Registered users are stored in Memory contain:

EnrollId , Username, HasFace , IsActive

Users are stored in:

```
public static ConcurrentDictionary<int, UserInfo> Users
```

which we will call RAM

AI Threshold Configuration

```
private static double _matchThreshold = 0.40;
```

```
private static float _livenessThreshold = 0.30f;
```

`_matchThreshold` is responsible for the similarity between the scan and stored data while `_livenessThreshold` is responsible for controlling spoof as in Fake Faces

AI Model Components

The System Loads 2 OpenCV-based models

```
private static FaceDetectorYN? _detector;
```

```
private static FaceRecognizerSF? _recognizer;
```

FaceDetectorYN

<https://github.com/panahandeh79/ArcFace-YuNet>

Detects faces inside the image.

FaceRecognizerSF

https://github.com/opencv/opencv_zoo/tree/main/models/face_recognition_sface

Generates **128-dimensional feature embeddings** for a face. These embeddings represent the facial structure mathematically.

Example:

[0.182, -0.043, 0.902, ... 128 values ...]

These vectors allow faces to be compared mathematically.

Model Initialization

```
public static void InitModels(...)
```

This function loads all AI models.

Loading Face Embeddings

```
public static void LoadEmbeddings()
```

This function loads all registered users from the database.

SQL Query

```
SELECT enrollid, username, record, isactive
```

```
FROM tblusers_face
```

```
WHERE backupnum = 50
```

record contains the face image encoded Base64.

Two-Phase Loading Strategy

Phase 1 — Fast Database Read

The system retrieves raw data; No AI processing happens.

Phase 2 – AI processing

Each image is processed:

1. Decode Base64 image
2. Detect face
3. Extract embedding vector
4. Store vector in memory

Result Stored in RAM

```
_knownEmbeddings → List<float[]>
```

```
_knownLabels → List<int>
```

Matching a Face

```
public static FaceMatchResult MatchFaceFromBase64(string probeBase64)
```

This function is called whenever a device sends a face scan.

Steps performed:

1. Decode Base64 image
2. Detect face
3. Extract embedding vector
4. Compare against known embeddings
5. Apply threshold rules
6. Return match result

Face Feature Extraction

```
GetFaceFeature(Mat input)
```

Steps performed internally:

- Resize image
- Detect face region
- Align the face
- Extract feature vector

Alignment ensures that faces are normalized before recognition.

Liveness Detection

```
float prob = AntiSpoofing.Predict(...)
```

Before matching, the system performs anti-spoofing detections to evaluate if the face is real or fake, if liveness score is below the threshold, the scan is rejected.

Face Matching Algorithm

The system compares embeddings using **cosine similarity**.

```
double Cosine(float[] a, float[] b)
```

Where:

- A = input face vector
- B = stored user vector

The system loops through all embeddings and the user with the **highest similarity score** becomes the match candidate. If the score exceeds the threshold, the match is accepted.

Match Result Object

The result returned:

```
FaceMatchResult
```

Example:

- Matched: true
- EnrollId: 1023
- Score: 0.72
- Liveness: 0.88

Adding a New User to Memory

```
AddUserToMemory()
```

When a new user enrolls:

- Face image is decoded
- Embedding is generated.
- Embedding is stored in RAM
- User info is added to Users

This avoids restarting the server after enrollment.

Removing a User

```
RemoveUserFromMemory()
```

This removes both:

- Embedding vector
- User info

Database Synchronization

```
SyncByComparisonAsync()
```

Every 30 seconds the system:

- fetches users from database
- compares them with RAM users
- adds missing users
- updates active status
- removes deleted users

This ensures RAM stays synchronized with the database.

Performance Considerations

To prevent AI conflicts, a lock protects the models

```
private static readonly object _aiLock
```

This prevents multiple threads from using the AI model simultaneously.

Antispoofing.cs — Liveness Detection Module

Purpose:

implements **liveness detection**, which determines whether the detected face is **real** or a **spoof attack**.

Spoof attacks can include:

- Printed Photo
- Phone Photo
- Video Photo
- Mask

Without anti-spoofing, attackers could bypass facial recognition using images.

This module uses a **deep learning model (MiniFASNet)** to detect fake faces.

<https://github.com/minivision-ai/Silent-Face-Anti-Spoofing/tree/master>

Core Mode

```
private static Net _net;
```

This neural network predicts whether a face is fake or not.

Model Initialization

```
public static void Init(string modelPath)
```

This function loads the anti-spoof model file from disk.

Steps performed:

1. Verify model file exists
2. Load the neural network
3. Log successful initialization

Prediction Process

```
public static float Predict(Mat originalImage, Rectangle faceBox)
```

This method analyses a detected face and returns the probability that the face is real.

Processing Steps

Crop Face Region – Step 1

The face area is expanded

```
CropWithScale(originalImage, faceBox, 2.7f)
```

This includes surrounding background information, which helps detect spoof artifacts.

Image Preprocessing – Step 2

The image is normalized before inference.

```
DnnInvoke.BlobFromImage()
```

Processing includes:

- resizing to 112×112
- converting BGR $\rightarrow$ RGB
- normalizing pixel values

Neural Network Inference – Step 3

```
net.Forward()
```

The neural network predicts whether the image is real or spoofed.

Softmax Normalization – Step 4

```
Softmax(scores)
```

The raw scores are converted into probabilities using the Softmax function. This converts model outputs into valid probabilities.

Output Result

Probability that face is REAL, If the value is below the configured threshold, the scan is rejected.

Thread Safety

```
private static readonly object _netLock
```

This ensures that multiple AI workers do not run inference simultaneously on the same model.

ScanJob.cs — Scan Task Structure

Purpose

a **single face scan request waiting to be processed by the AI engine**.

These jobs are placed into the **AI scan queue** inside WebSocketLoader.

The queue allows scans to be processed asynchronously by worker threads.

Class Definition

```
public sealed class ScanJob
```

This class stores the data required to process a face scan

Fields

Device Serial Number

```
public required string DeviceSn
```

Identifies which device sent the scan.

Face Image

```
public required string ImageBase64
```

The captured face image encoded as Base64

WebSocket Session

```
public required WebSocketSession Session
```

This allows the system to send a response back to the device.

Timestamp

```
public required DateTime Timestamp
```

Stores the time the scan was received for logging and debugging

ConsoleInteractor.cs — Console Interaction Utility

Purpose

Provides **non-blocking console input handling**.

Normally, reading input from the console blocks the program.

This class allows the server to:

- Ask a question
- Wait for input
- Continue processing if no response is received

Key Feature

It uses a **TaskCompletionSource** to manage asynchronous input.

```
TaskCompletionSource<string?>
```

This allows the system to wait for user input without stopping the server.

Ask a Question

```
AskAsync(prompt, timeout)
```

If the user does not respond within the timeout period, the system continues automatically.

Timeout Handling

```
cts.CancelAfter(timeout)
```

A cancellation token is used to stop waiting after a defined time.

Yes / No Helper

```
AskYesNoAsync()
```

This simplifies confirmation prompts.

Thread Safety

```
lock(_lock)
```

The class ensures that only one active console question exists at a time preventing conflicting console inputs

WebSocketService.cs — Windows Service Wrapper

Purpose

allows the server to run as a **Windows Service** which makes it able to run in the background and start automatically with the operating system.

This allows the system to run **continuously without manual execution**.

Linux Server Implementation

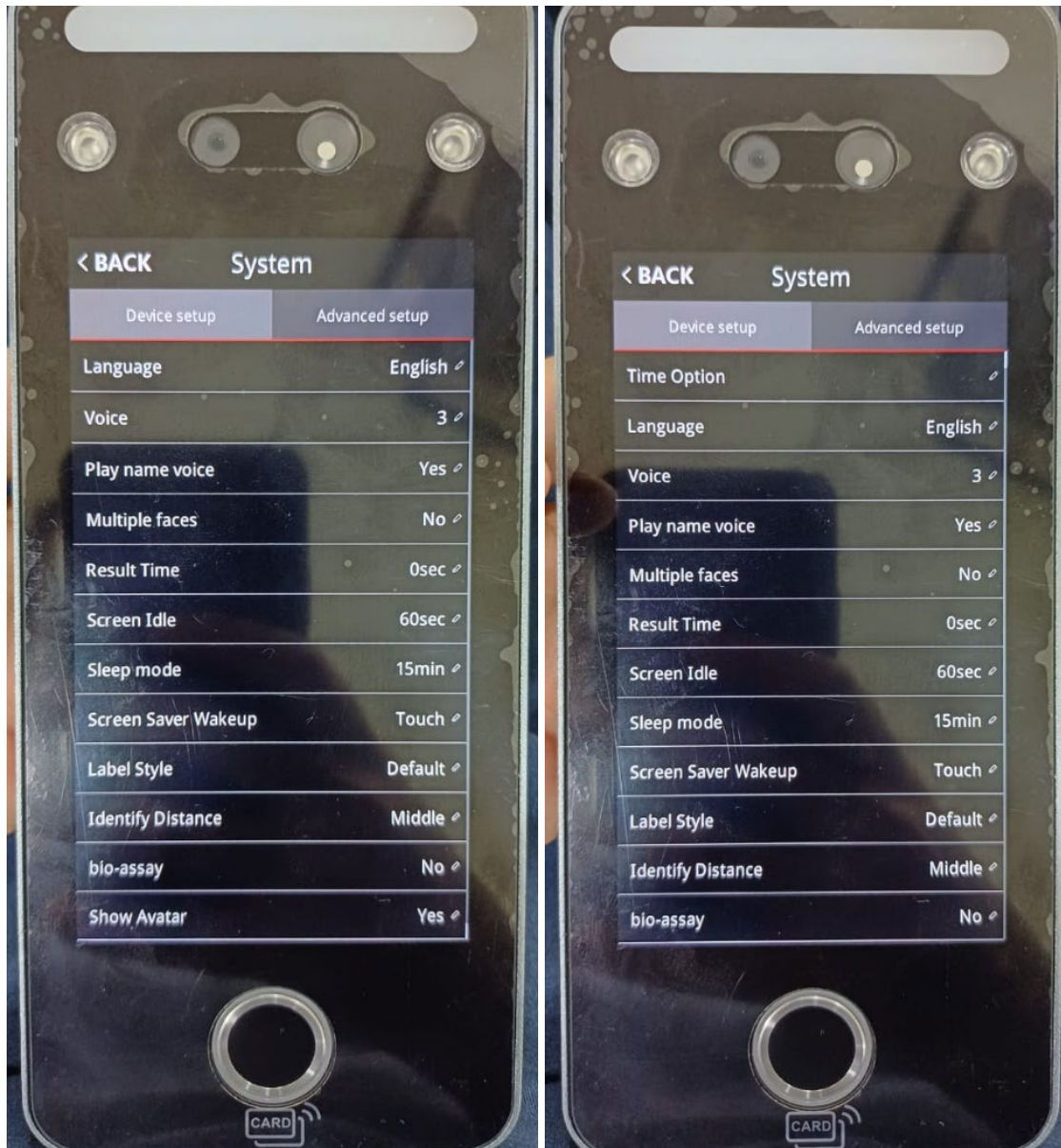
As this server will be mainly run on Linux Ubuntu Server, This File Can be Ignored unless needed.

Device Settings

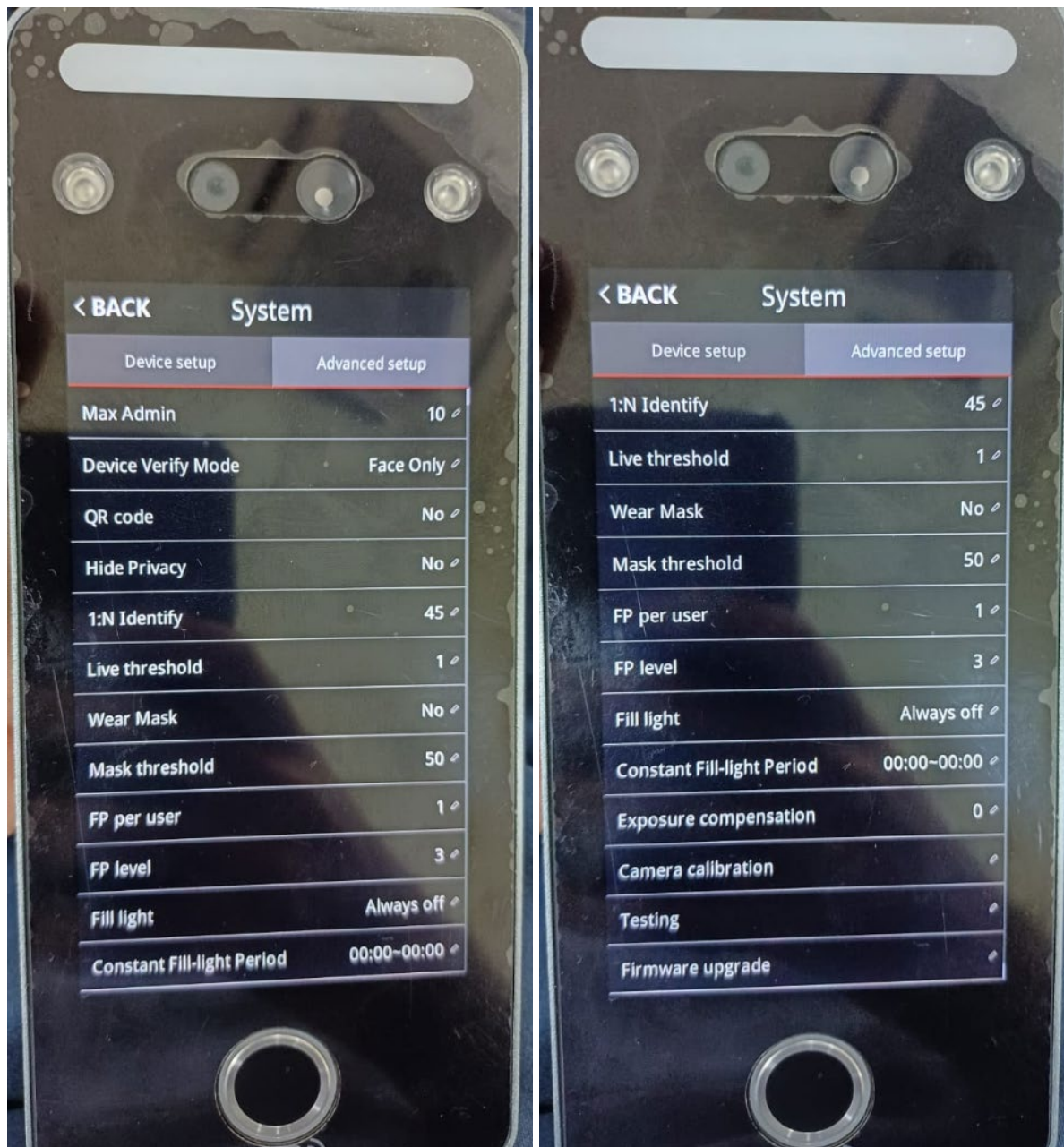
Best AI07F Device Settings that work with the server with no issues.

System:

Device Setup:



Advanced Setup



Access:

< BACK	Access
OD delay	1sec ↗
Sensor Mode	NG ↗
Sensor Delay	0sec ↗
Stranger Access	No ↗
Door Password	* ↗
Users	0 ↗
Anti pass	No ↗
Wg output	ID ↗
Wg format	34bit ↗
Card Disp Format	Digital ↗
External Wiegand Snaf	No ↗
Tamper alarm	No ↗
Inter lock	No ↗

< BACK	Access
Users	0 ↗
Anti pass	No ↗
Wg output	ID ↗
Wg format	34bit ↗
Card Disp Format	Digital ↗
External Wiegand Snaf	No ↗
Tamper alarm	No ↗
Inter lock	No ↗
Alarm proc	No ↗
Fail count	0 ↗
Punch times in timezo	0 ↗
Access denied no LOG	No ↗
Out of NO TimeZone D	No ↗

Shift:

< BACK		Shift	
Atten rules	Edit shift	Bell	Log Info
Re verify		5sec	
Download shift			
Upload shift			
Log Warning		1000	
Save Photo		No	
Stranger Photo		Yes	
Verify fail Hint		Yes	
Self-service check log		Yes	
Default Shift		shift 1	
Weekend		Sat.&Sun.off	
Weekend Overtime		No	
Auto sign		Sign all	

< BACK		Shift	
Atten rules	Edit shift	Bell	Log Info
Save Photo		No	
Stranger Photo		Yes	
Verify fail Hint		Yes	
Self-service check log		Yes	
Default Shift		shift 1	
Weekend		Sat.&Sun.off	
Weekend Overtime		No	
Auto sign		Sign all	
Excel Pwd			
Late time		0min	
Lea time		0min	
Reverify not Open doo		No	